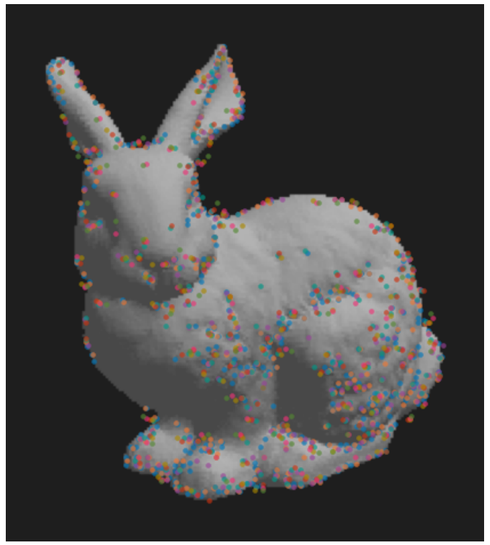
1. Which points do we keep?

FAST finds possible corner points. We then decide which of those points to keep.

Here, both selection rules keep 1313 points. They choose slightly different points.

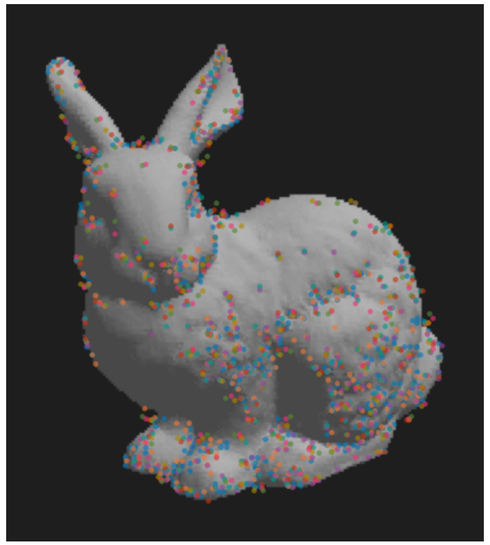
1. Selected using FAST

Points chosen by their FAST scores



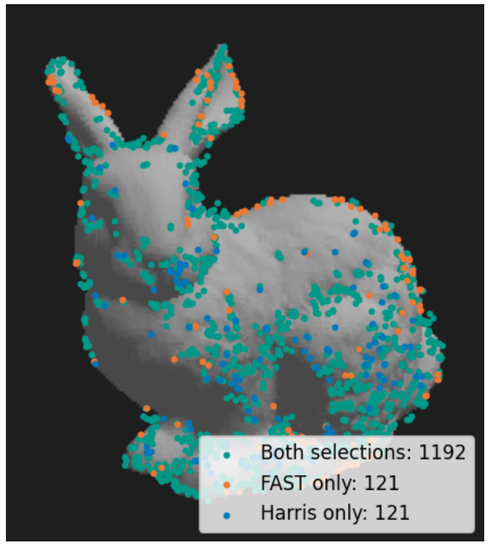
2. Selected using Harris

Points chosen by their Harris scores



3. Compare the two

See which selected points differ



Green: 1192 points kept by both rules.

Orange: 121 kept only by FAST. Blue: 121 kept only by Harris.

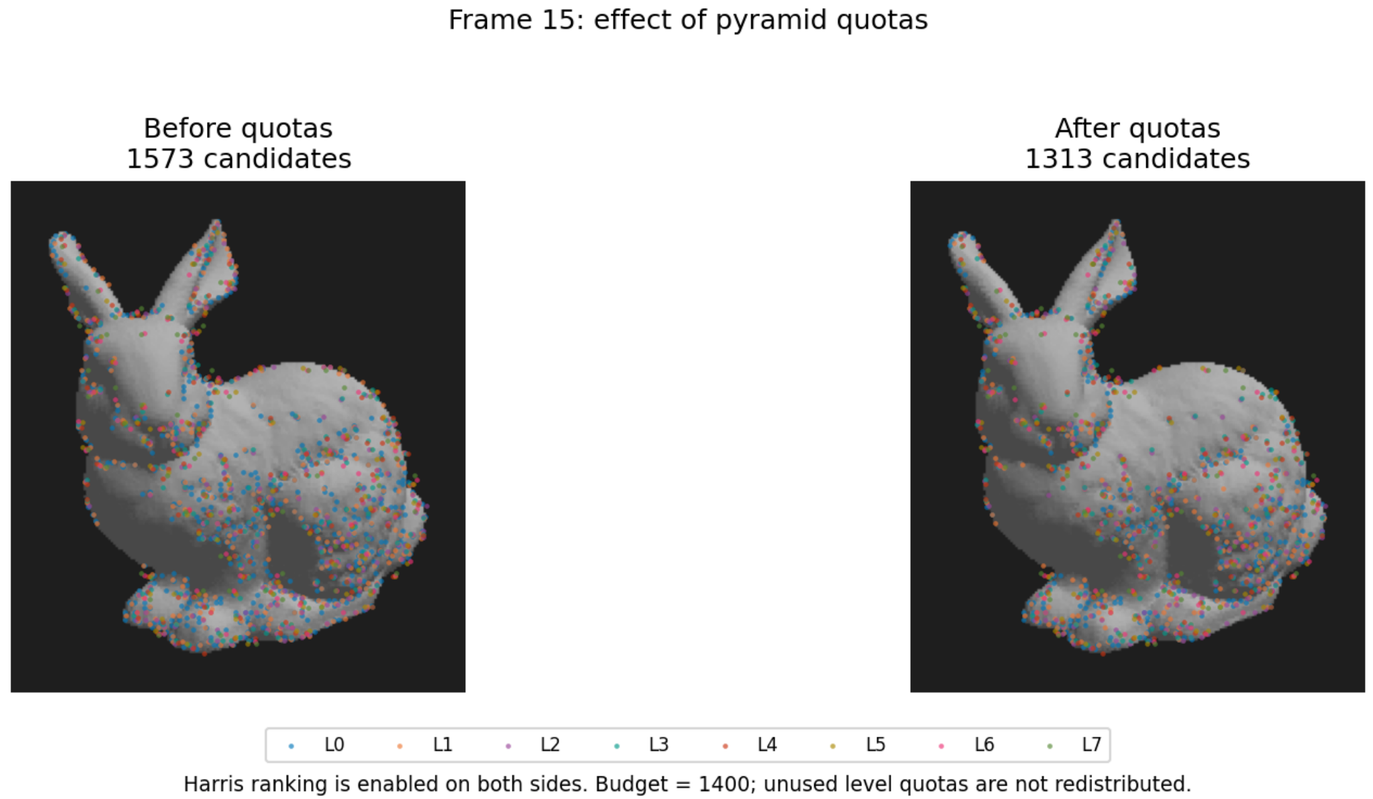
Harris replaces 121 selected points with 121 different ones. It does not move the points.

Image source: panels from outputs/frontend/pair_15_16/frame_000015_03_ranking_same_quota.png

2. Limit points at each image size

We use 8 versions of each picture: the original and seven smaller versions.

Each version can keep only a set number of points. This limit is called its quota.



Example: a limit of 304 means keep at most 304 points from that image size. Here, 1573 detected points become 1313 selected points. Some versions have fewer points than their limit.

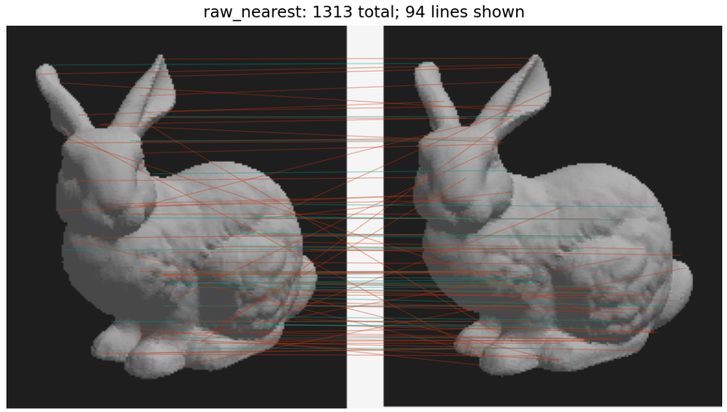
Image source: outputs/frontend/pair_15_16/frame_000015_02_quotas_before_after.png

3. Check matches between two pictures

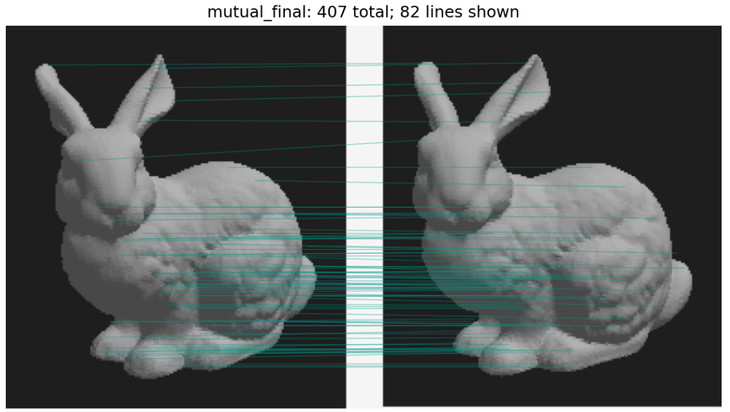
A point in the first picture may look similar to several points in the next picture.

We keep a match only if it is clearly better than the alternatives and the reverse match agrees.

Before: 1313 suggested matches



After: 407 matches kept



The ratio compares the best match with the next-best match. If both look nearly equally good, we reject the match. The next slide gives an example.

1313 suggestions -> 730 pass the first check -> 407 pass both directions. The code checks their 3D consistency later.

Image source: outputs/frontend/pair_15_16/matches_raw_nearest.png and matches_mutual_final.png

What does the ratio check do?

For one point, compare the two most similar points in the next picture. The code gives each a difference score: smaller means more similar.

Ratio = best difference score / next-best difference score

Example A: the best match is much clearer

Scores: 20 and 50. Ratio = 20 / 50 = 0.40. This passes the 0.87 limit.

Example B: the two matches are too similar

Scores: 40 and 42. Ratio = 40 / 42 = 0.95. This fails the 0.87 limit.

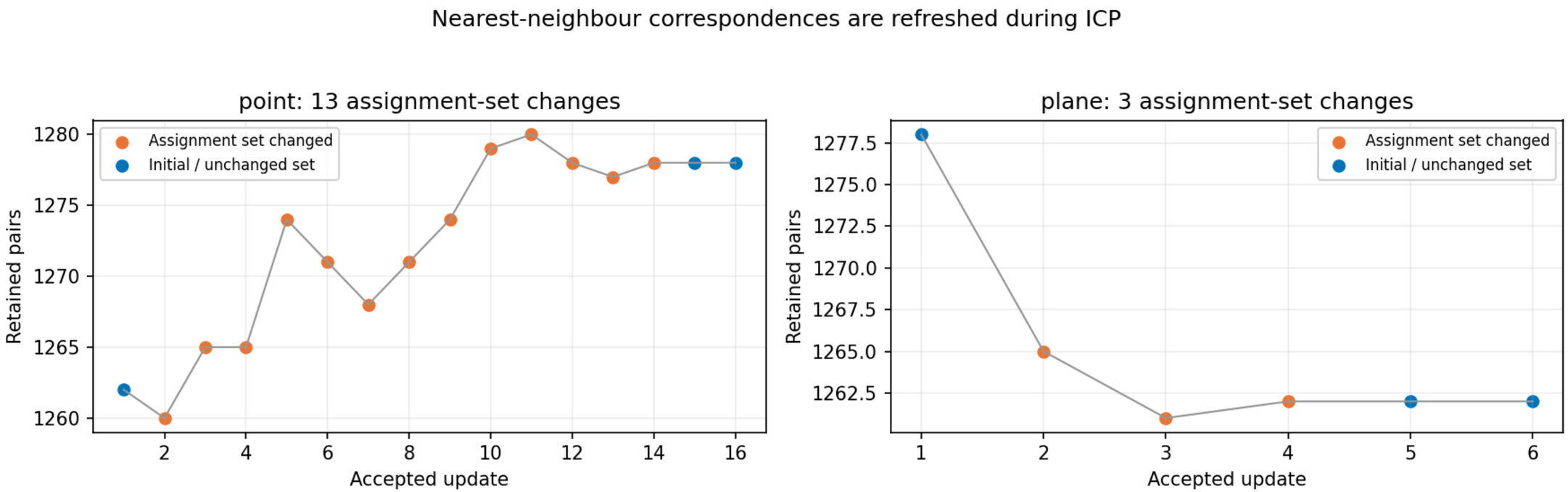
Then check the other direction: if point A chooses point B, point B must choose A too. The reverse ratio must also pass.

The ratio is a comparison score, not the probability that a match is correct.

4. Find the closest 3D points again

ICP moves one set of 3D points to line up with the other set.

After each move, the closest point may change. We search again instead of keeping old pairs.



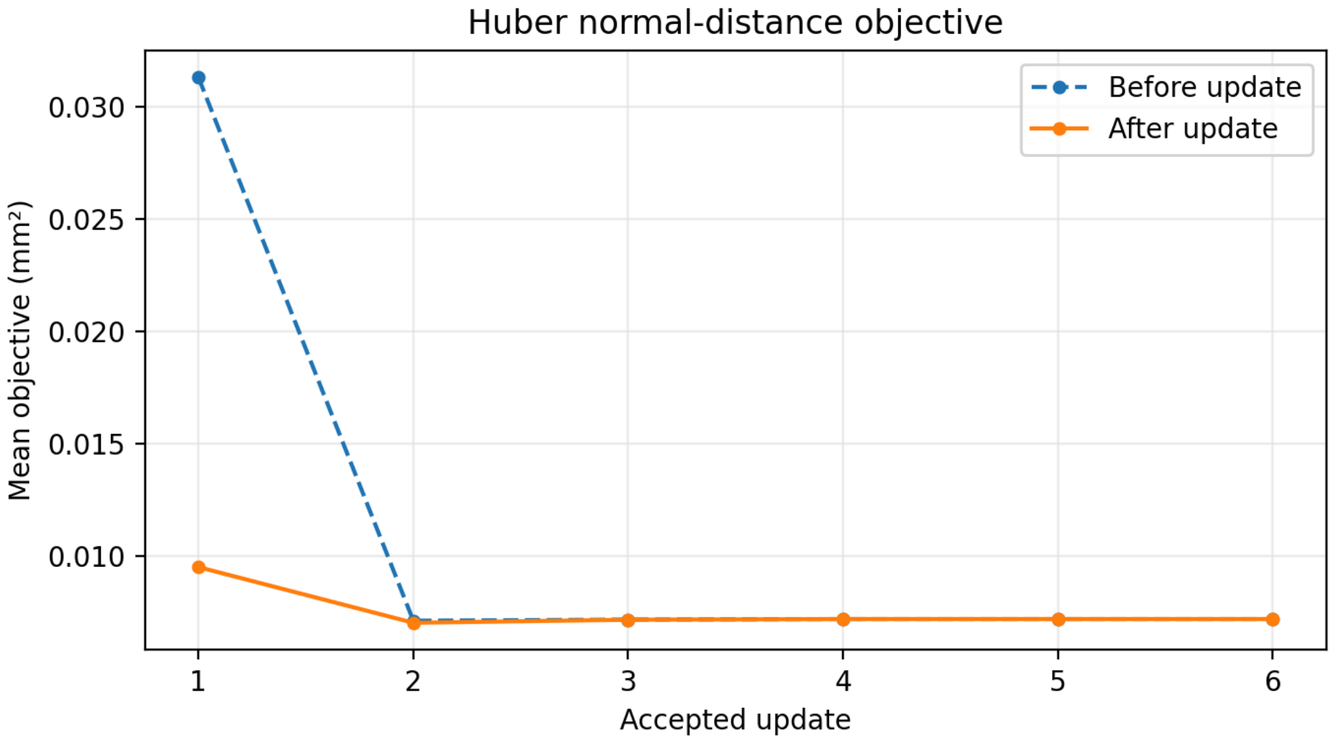
For frames 15 and 16, the code makes 16 point-alignment updates. The set of point pairs changes 13 times. Across the full sequence, position RMSE falls from 156.31 mm to 45.82 mm.

Image source: outputs/icp/04_nearest_neighbour_refresh.png

5. Use the surface to refine alignment

Treat each small surface patch as a tiny flat sheet, then move points towards that sheet.

This extra step reduces camera-position RMSE from 45.82 mm to 1.69 mm.



45.82 mm

point ICP

to

1.69 mm

plane ICP

The graph shows the surface-alignment error becoming smaller. This step measures distance to a surface, which differs from distance to a single point.

Source: icp_cpp_results/iteration_history.csv, plane GN objective for frames 15-16

Measured improvement

Current ICP stages

Method	Position RMSE
Feature-pair SVD	245.49 mm
RANSAC initialization	156.31 mm
Point-to-point ICP	45.82 mm
Point-to-plane ICP	1.69 mm
Verified loop + graph	1.20 mm

Scratch PnP: Previous vs Updated

Metric	Previous	Updated
Mean relative rotation	1.16 deg	0.484 deg
Mean relative translation	10.7 mm	4.3 mm
Final position error	284 mm	40.46 mm
Final rotation error	31.9 deg	3.86 deg

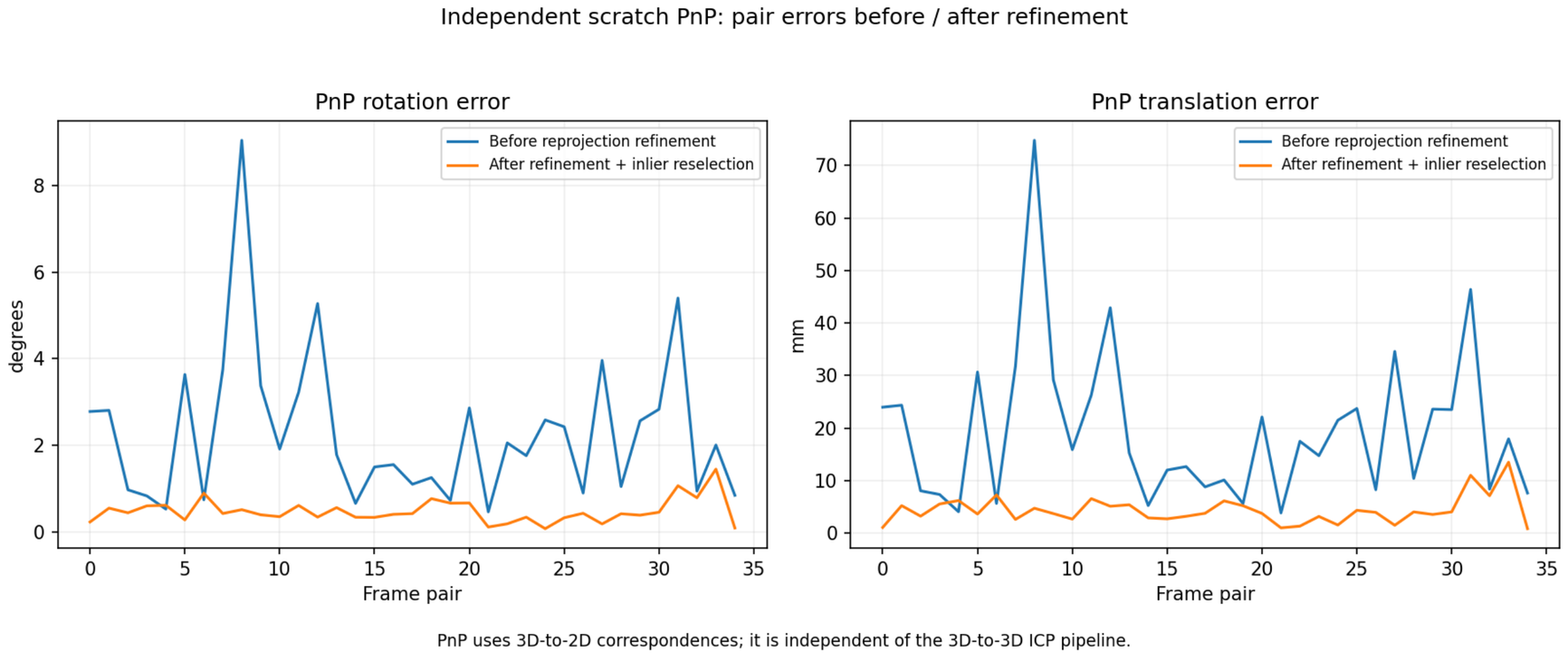
Both methods process all 35 frame pairs. ICP also recognizes a return to an earlier location and uses that connection to adjust the camera path.

PnP improves after several changes together: point selection, depth checks and pose refinement. We cannot say Harris alone caused this gain. PnP still drifts and does not get the ICP loop correction.

Sources: supplied slides, page 12; outputs/icp/summary_metrics.csv; current scratch pipeline summary and trajectory.

Other improvements

We also remove unreliable depth points, reject matches that disagree with the estimated motion, and refine PnP. A verified return to an earlier location helps correct ICP drift.



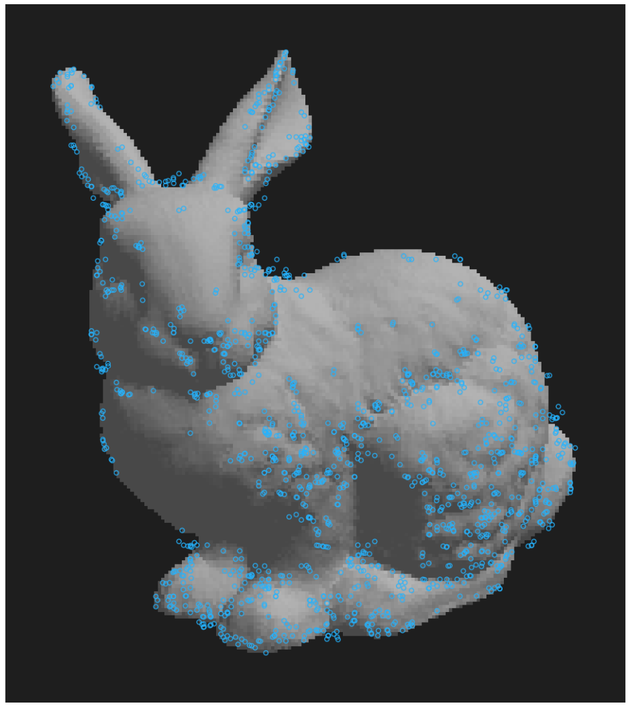
Harris scores are calculated once per point to save work. The results are saved as PNGs for easy viewing. This latest display update does not change the estimated camera positions.

Image source: outputs/icp/06_pnp_refinement_before_after.png

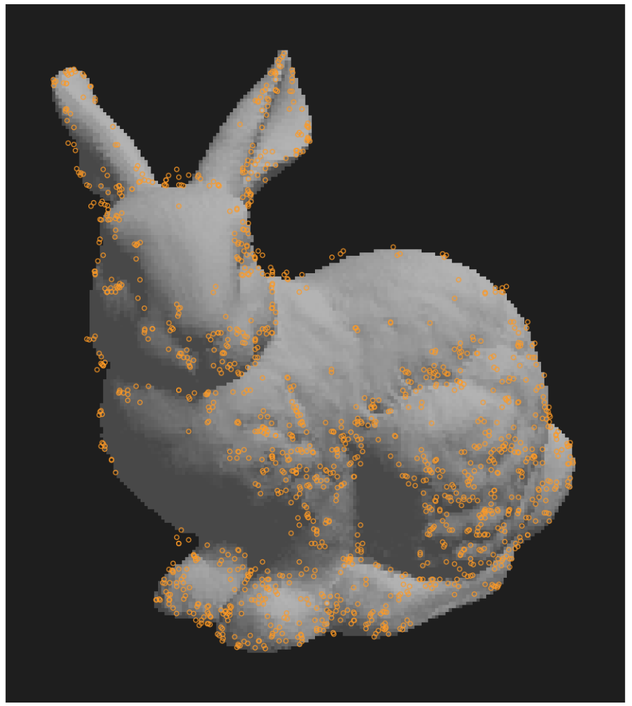
Our keypoints and OpenCV ORB

Frame 15, all eight pyramid levels. Each panel uses the same image crop and magnification.

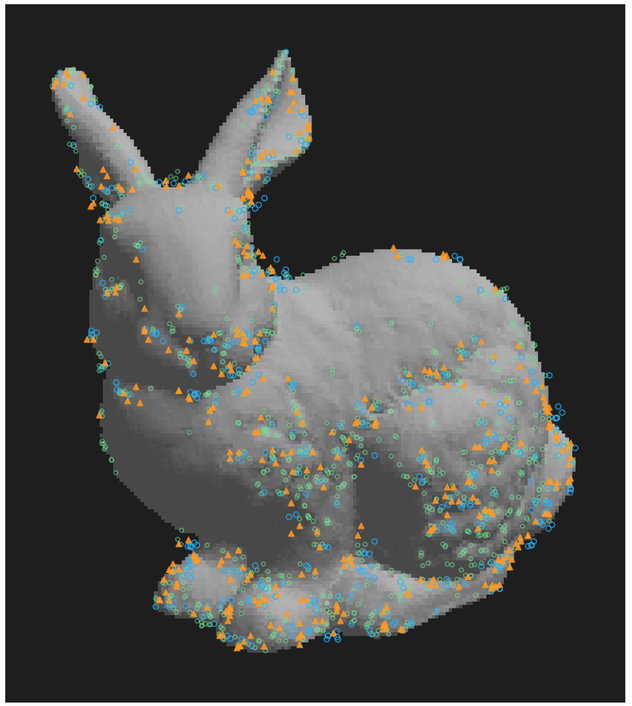
Our extractor: 1313 points



OpenCV ORB: 1292 points



Spatial overlap and differences



932 spatial pairs. 381 of our points and 360 OpenCV points remain unpaired.

Difference panel: blue circles are ours only, orange triangles OpenCV only, green nearby pairs.

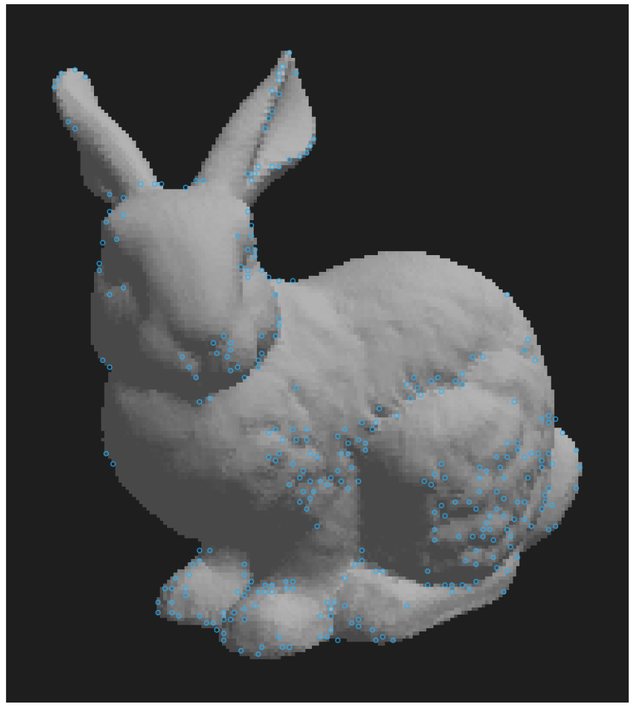
Overlap rule: one-to-one greedy pairing within 2 original-image pixels at the same pyramid level. Spatial overlap only.

Source: exact C++ keypoint exports from the benchmark. All retained points shown. No cross-method descriptor test.

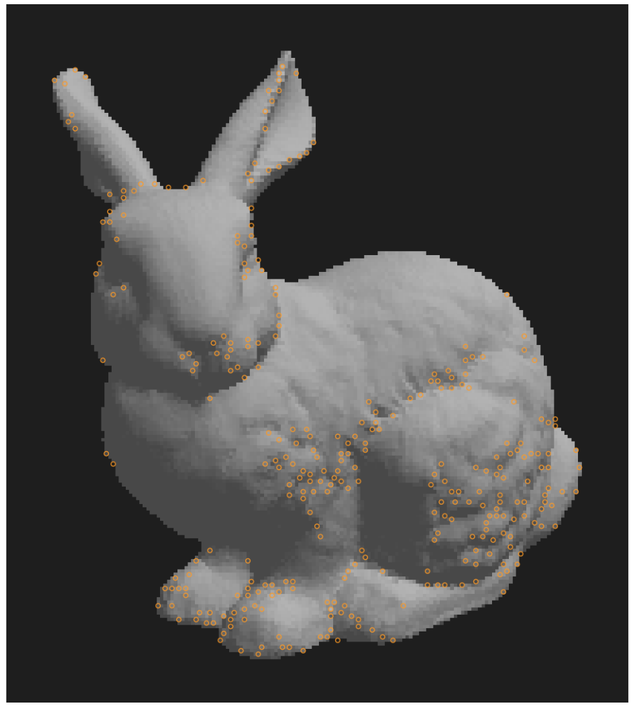
Keypoint differences at the same image scale

Frame 15, pyramid level 0 only. Equal feature counts can still select different pixel locations.

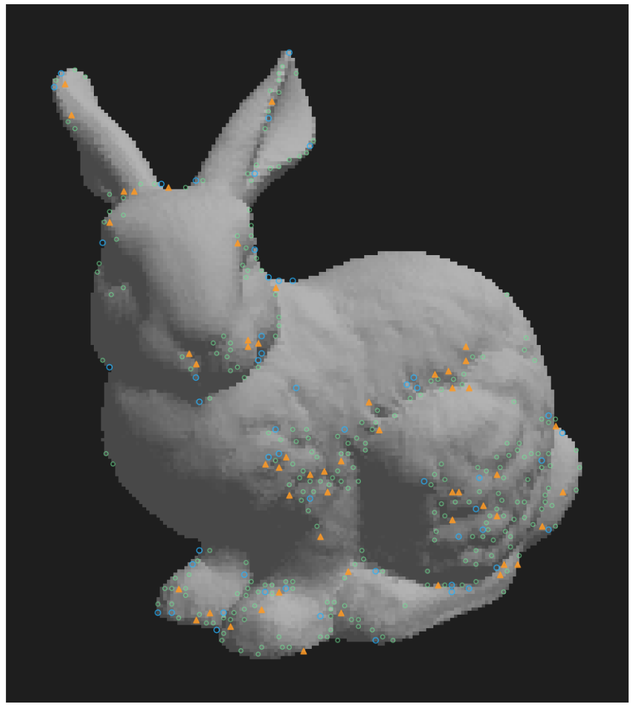
Our extractor: 304 points



OpenCV ORB: 304 points



Spatial overlap and differences



251 spatial pairs. 53 of our points and 53 OpenCV points remain unpaired.

Difference panel: blue circles are ours only, orange triangles OpenCV only, green nearby pairs.

Overlap rule: one-to-one greedy pairing within 2 original-image pixels at the same pyramid level. Spatial overlap only.

Source: exact C++ keypoint exports from the benchmark. All retained points shown. No cross-method descriptor test.

Basic Gauss-Newton pose optimization

We adjust one camera pose: 3 rotation variables and 3 translation variables. The 3D clouds stay fixed.

1. Measure the alignment error

$$x_i = R\,p_i + t$$

$$r_i = x_i - q_i$$

$$E = \tfrac{1}{2} \sum_i \|r_i\|^2$$

p_i: source point. q_i: its current target neighbour.

A residual gives the remaining x, y and z errors.

2. Approximate a small correction

$$\delta = [\omega^T \;\; v^T]^T \quad \text{(six pose variables)}$$

$$x_i(\text{new}) \approx x_i + \omega \times x_i + v$$

$$r_i(\text{new}) \approx r_i + J_i\,\delta$$

$$J_i = [-[x_i]\times \quad I] \quad (3 \times 6)$$

Each column predicts the effect of one variable.

3. Solve for the correction that reduces the predicted error

$$H = \sum_i J_i^T J_i, \quad b = -\sum_i J_i^T r_i$$

$$\mathbf{(H + \lambda I)\,\delta = b}$$

The code solves this 6 × 6 system with LDLT. Small diagonal damping λ stabilizes the solve.

Simple example: point = 2.0, target = 2.3. Residual = −0.3 and J = 1.

The undamped translation correction is +0.3, moving the point to 2.3.

Gauss-Newton uses a local linear approximation of the residual. It repeats these small corrections.

Code: pointJacobian(), pointICP() in src/tracking/rgbd_icp.cpp. Units: rotations in radians, translations in metres.

Gauss-Newton updates in our ICP code

We use basic damped Gauss-Newton with backtracking. Plane ICP also uses robust Huber weights.

Point-to-point update

$\lambda = 10^{-5}\,\mathrm{trace}(H) / 6 + 10^{-9}$

Solve $(H + \lambda I)\,\delta = b$

Bound the step: rotation ≤ 0.1 rad, shift ≤ 0.03 m.

Try $\alpha = 1, \tfrac{1}{2}, \tfrac{1}{4}, \tfrac{1}{8}, 1/16$.

$R(\mathrm{new}) = \mathrm{Exp}(\alpha\omega)\,R$

$t(\mathrm{new}) = \mathrm{Exp}(\alpha\omega)\,t + \alpha v$

Accept a safe step that decreases fixed-pair error.

Find neighbours again for the next iteration.

Point-to-plane refinement

$e_i = n_i^T(R\,p_i + t - q_i)$

$j_i = n_i^T J_i$

$w_i = 1$ if $|e_i| \le h$, else $h / |e_i|$

$h = 0.004$ m (4 mm)

$H = \Sigma_i\, w_i\, j_i^T j_i$

$b = -\Sigma_i\, w_i\, j_i^T e_i$

Use the same solve and pose-update steps.

Normals measure motion towards each local surface.

Frames 15-16: 16 point updates and 6 plane updates. Neighbour sets change during both stages.

RMSE: point ICP 45.82 mm, plane ICP 1.688 mm, loop + graph 1.200 mm.

Only the six pose variables change in each ICP solve. RANSAC still supplies the initial pose using SVD.

The update uses a left $SO(3) \times R^3$ retraction. The ICP solve uses no adaptive Levenberg-Marquardt schedule.

Code: pointICP(), planeICP(), leftIncrement(). Limits: 18 point iterations and 16 plane iterations per registration.

ORB comparison and Gauss-Newton ICP

36 RGB-D frames. Both extractors use the same matching, depth checks and GN ICP.

ORB frontend comparison

Metric	Project ORB-style	OpenCV ORB
Solved / failed pairs	35 / 0	35 / 0
Mean features / frame	1013.9	993.9
Mean matches / pair	275.6	286.3
Mean valid 3D / pair	214.5	213.2
Mean RANSAC inliers / pair	208.5	207.7
GT match precision	97.00%	97.07%
Median extraction (ms/frame)	81.345	4.405
Mean matching (ms/pair)	170.324	213.802
Mean registration (ms/pair)	190.239	236.306
Trajectory RMSE (mm)	1.688	1.692
Mean relative rotation (deg)	0.02800	0.02803
Mean relative translation (mm)	0.204	0.204

Gauss-Newton pose update

Hold the point pairs fixed during one update.

$r_i = R\,p_i + t - q_i, \quad E = \tfrac{1}{2}\,\Sigma_i \,\|r_i\|^2$

$\delta = [\omega^T \;\; v^T]^T$ (3 rotation + 3 translation)

$r_i(\text{new}) \approx r_i + J_i\,\delta$

$J_i = [-[x_i]\times \quad I], \quad x_i = R\,p_i + t$

$H = \Sigma_i\, J_i^T J_i, \quad b = -\Sigma_i\, J_i^T r_i$

$(H + \lambda I)\,\delta = b$ (a 6 × 6 system)

$R(\text{new}) = \text{Exp}(\omega)\,R$

$t(\text{new}) = \text{Exp}(\omega)\,t + v$

Accept a descending step, refresh neighbours, repeat.

Plane ICP: $e_i = n_i^T r_i, \;\; j_i = n_i^T J_i$

$H = \Sigma_i\, w_i\, j_i^T j_i, \;\; b = -\Sigma_i\, w_i\, j_i^T e_i$ (Huber w_i)

Controls: 1400-feature budget, 8 levels, scale 1.2, FAST 10, mutual Hamming ratio 0.87. Times depend on hardware.

Current GN run: point ICP 45.82 mm, plane ICP 1.688 mm, loop + graph 1.200 mm. ICP changes one six-variable pose per update.

GT precision: 9 mm 3D threshold. Registration includes matching. ORB comparison uses no loop closure. Blender: 1 unit = 1 m.

Sources: outputs/orb_benchmark/summary.csv; icp_cpp_results/summary_metrics.csv; src/tracking/rgbd_icp.cpp